

End-to-End Batch ETL Pipeline for NYC Taxi Trip Data

PySpark • Apache Airflow • PostgreSQL • GitHub Actions CI

Author	Pooja Hasurkar
Role	Data Engineer
Repository	github.com/hasurkarpooja/mini-etl-pipeline
Stack	PySpark, Apache Airflow, PostgreSQL, Docker, GitHub Actions

Overview

This project is a self-directed portfolio build of a production-style batch data engineering pipeline, designed to mirror the real-world ETL patterns used in enterprise data migration work (PySpark, Databricks-style orchestration, and warehouse loading). The goal was to demonstrate, in a public and verifiable repository, the same engineering practices used in professional projects: modular pipeline design, orchestration, automated testing, and continuous integration.

Problem Statement

Raw NYC Yellow Taxi trip records are published monthly as large parquet files containing malformed and duplicate rows, invalid fares, and inconsistent timestamps. The objective was to build a repeatable pipeline that ingests this raw data, cleans and enriches it at scale, and lands query-ready aggregate tables in a relational warehouse — on a schedule, with retry logic and monitoring, rather than as a one-off notebook exercise.

Architecture & Approach

The pipeline follows a standard three-stage batch ETL pattern, orchestrated end-to-end by an Airflow DAG:

- **Extract** — downloads a monthly trip-record parquet file from the public NYC TLC dataset into a raw storage zone, skipping re-download if the file already exists.
- **Transform** — a PySpark job that deduplicates records, filters out invalid fares and trip durations, derives fields such as pickup hour and weekday, and aggregates average fare and trip duration by pickup zone and hour.
- **Load** — writes the cleaned fact table and hourly aggregate table into PostgreSQL using batched upserts, making the load step idempotent and safe to re-run.
- **Orchestration** — an Airflow DAG chains the three stages with a monthly schedule, automatic retries on failure, and failure alerting.

Tech Stack

Layer	Tool
Processing	PySpark
Orchestration	Apache Airflow
Raw storage	Parquet
Curated storage	PostgreSQL
Local dev environment	Docker Compose
CI / automated testing	GitHub Actions, pytest, flake8

Engineering Practices Applied

- **Modular codebase** — extract, transform, load, and configuration are separated into distinct modules rather than a single script, matching how the logic would be maintained in a team setting.
- **Idempotent loads** — the load stage uses upserts keyed on zone and hour, so re-running the pipeline for a given period does not create duplicate data.
- **Automated data quality filtering** — invalid fares and implausible trip durations are filtered out during transform based on configurable thresholds.
- **Unit testing** — transform logic is covered by pytest tests using an in-memory Spark session, validating filtering and aggregation behavior independent of real data.
- **Continuous Integration** — every push triggers a GitHub Actions workflow that lints the codebase with flake8 and runs the full test suite before code is considered mergeable.
- **Version control discipline** — work was committed incrementally with descriptive messages, rather than as a single bulk upload, mirroring a real development workflow.

Challenge: Debugging a Failing CI Pipeline

On the first push, the GitHub Actions CI workflow failed at the lint stage. Rather than disabling the check, each failure was diagnosed and resolved individually across three follow-up commits:

- **E402 (module-level import not at top of file)** — caused by imports that depend on a runtime `sys.path` modification; resolved by explicitly marking those imports as intentional.
- **E302 (expected 2 blank lines)** — a PEP 8 spacing violation before a function definition; resolved by correcting blank-line formatting.
- **E303 (too many blank lines)** — an overcorrection from the previous fix; resolved by matching the exact PEP 8 spacing convention.

This iterative debugging is reflected directly in the repository's commit history, and the pipeline now runs green on every push.

Outcome

The result is a fully public, runnable, and tested batch ETL pipeline that processes taxi trip data end-to-end, with a green CI badge, unit test coverage of core transformation logic, and a commit history that documents real engineering iteration rather than a single code dump.

Skills Demonstrated

- Batch data pipeline design and PySpark-based transformation logic
- Workflow orchestration with Apache Airflow (scheduling, retries, task dependencies)
- Relational data loading and idempotent upsert patterns in PostgreSQL
- Automated testing of Spark transformation logic with pytest
- CI/CD pipeline configuration and debugging with GitHub Actions
- Git version control: incremental commits, troubleshooting, and iteration

Repository: github.com/hasurkarpooja/mini-etl-pipeline | LinkedIn: linkedin.com/in/pooja-hasurkar